ONLINE INTERNSHIP MANAGEMENT PORTAL

A full-stack platform connecting Students, Companies and Administrators

HTML • CSS • JavaScript | Node.js • Express.js | MongoDB

Project status: In progress — architecture and core backend structure established

1. Problem Statement & Proposed Solution

Current Problems

- Students depend on forms, emails and scattered internship information.
- Companies handle applications and candidate tracking manually.
- Colleges have limited visibility into internship progress.
- Reports, approvals and certificates are difficult to organize.
- Communication and status tracking are fragmented.

Our Solution

- One centralized portal for the complete internship lifecycle.
- Separate dashboards for Student, Company and Admin roles.
- Digital internship discovery and application tracking.
- Report submission, verification and certificate management.
- Secure APIs, role-based access and MongoDB persistence.

2. Users, Modules & Key Features

Student

Register / Login
Manage profile & resume
Browse internships
Apply and track status
Upload internship reports
View/download certificates

Company

Register / Login
Create internship postings
Edit and manage postings
View applicants
Accept / reject applications
Track interns

Admin

Manage students and companies
Approve/monitor internships
Review reports
Manage certificates
View analytics and reports
Maintain system controls

3. Project Analysis Blueprint

InternManage
Internship Management Portal

- Project Analysis
- Problem Statement
- Users & Roles
- Requirements
- Use Cases
- Database Design
- API Analysis
- UI Analysis
- Tech Stack
- Project Flow



Well planned projects build successful systems.

Project Analysis

Online Internship Management Portal

A complete platform that connects students, companies, and institutions to manage internship opportunities, applications, reports and certificates in one place.



1. Problem Statement

- Students apply through emails or forms.
- Companies manage applications manually.
- Colleges cannot track progress efficiently.
- Reports and certificates are hard to manage.

2. Our Solution

- Centralized platform for all internship activities.
- Role-based dashboards for students, companies & admins.
- Easy application tracking and progress monitoring.
- Digital reports and certificate management.

3. Users & Roles

Student
- Register / Login
- Manage Profile & Resume
- Browse Internships
- Apply & Track Applications
- Upload Reports
- Download Certificates

Company
- Register / Login
- Post Internships
- Manage Internships
- View & Shortlist Applicants
- Accept / Reject Applications
- Track Intern Progress

Admin
- Manage Students & Companies
- Approve Companies
- Manage Internships
- Monitor Applications & Reports
- Generate Reports & Analytics
- Manage Certificates

4. Functional Requirements

01	Authentication (Login / Register)
02	Role-based Dashboards
03	Internship Management
04	Application Management
05	Report Submission
06	Certificate Generation
07	Notifications & Alerts
08	Admin Analytics & Reports

5. Non-Functional Requirements

- Secure & authenticated system
- Fast and responsive
- Easy to use interface
- Scalable and maintainable
- Data backup and recovery
- Cross-platform (Web & Mobile)

6. Use Cases (High Level)



7. Database Design (Collections)



8. API Analysis (Sample Endpoints)

POST	/api/auth/register	Register a new user
POST	/api/auth/login	User login
GET	/api/internships	Get all internships
POST	/api/internships	Create new internship
GET	/api/internships/:id	Get internship details
POST	/api/applications	Apply for internship
GET	/api/applications/my	Get my applications
PUT	/api/applications/:id	Update application status
POST	/api/reports	Upload internship report
GET	/api/certificates/:id	Download certificate

9. UI Analysis (Pages)

Public Pages
Home
About
Features
How It Works
Contact

Student Pages
Dashboard
Profile
Browse Internships
Internship Details
My Applications
Upload Report

Company Pages
Dashboard
Post Internship
Manage Internships
Applicants
Company Profile

Admin Pages
Dashboard
Students
Companies
Internships
Reports
Analytics

10. Project Flow



11. Technology Stack

- **Frontend** — React.js, HTML5, CSS3, Tailwind CSS
- **Backend** — Node.js, Express.js
- **Database** — MongoDB
- **Authentication** — JWT, bcrypt
- **Other Tools** — Multer, Cloudinary, Postman, Git, GitHub

4. Overall System Architecture

Frontend

HTML + CSS + JavaScript

Server

Node.js + Express.js

Routes

REST API endpoints

Controllers

Business logic

Models

Mongoose schemas

Database

MongoDB

Middleware Layer

Authentication • Authorization • Validation • Error handling

5. MongoDB Database Design

InternManage
Internship Management Portal

Database Design
MongoDB Schema

The database is designed using MongoDB to store and manage all entities of the internship management system efficiently.

NoSQL Database
MongoDB is a document oriented NoSQL database which stores data in JSON-like documents.

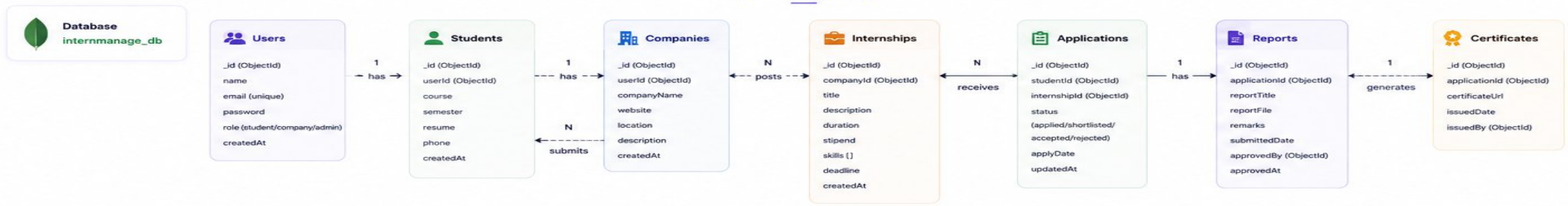
Flexible Schema
Easy to update schema without affecting the existing data or application.

Scalable
Easily scalable horizontally for growing data and high traffic.

High Performance
Optimized for faster read/write operations and aggregation queries.

Secure
Data validation, authentication, and role-based access ensure security.

Entity Relationship Diagram



Collection Schemas (Sample Documents)

users

```
{
 "_id": ObjectId("64f8a..."),
 "name": "Niramay Patanwadiya",
 "email": "niramay@example.com",
 "password": "hashed_password",
 "role": "student",
 "createdAt": ISODate("2024-05-20T10:00:00Z")
}
```

students

```
{
 "_id": ObjectId("64f8b..."),
 "userId": ObjectId("64f8a..."),
 "course": "BCA",
 "semester": 6,
 "resume": "resumes/niramay.pdf",
 "phone": "9876543210",
 "createdAt": ISODate("2024-05-20T10:05:00Z")
}
```

companies

```
{
 "_id": ObjectId("64f8c..."),
 "userId": ObjectId("64f8b..."),
 "companyName": "TechSoft Pvt. Ltd.",
 "website": "https://techsoft.com",
 "location": "Ahmedabad, Gujarat",
 "description": "Software Development
  Company",
 "createdAt": ISODate("2024-05-21T09:30:00Z")
}
```

internships

```
{
 "_id": ObjectId("64f8e..."),
 "companyId": ObjectId("64f8c..."),
 "title": "Web Development Intern",
 "description": "Work on real world
  projects using MERN stack.",
 "duration": "2 Months",
 "stipend": 8000,
 "skills": ["HTML", "CSS", "JavaScript", "React"],
 "deadline": ISODate("2024-06-30"),
 "createdAt": ISODate("2024-05-21T09:45:00Z")
}
```

applications

```
{
 "_id": ObjectId("64f8f..."),
 "studentId": ObjectId("64f8c..."),
 "internshipId": ObjectId("64f8e..."),
 "status": "applied",
 "applyDate": ISODate("2024-05-25"),
 "updatedAt": ISODate("2024-05-25T12:00:00Z")
}
```

reports

```
{
 "_id": ObjectId("64f90..."),
 "applicationId": ObjectId("64f8f..."),
 "reportTitle": "Final Report",
 "reportFile": "reports/final_report.pdf",
 "remarks": "Good Work",
 "submittedDate": ISODate("2024-07-10"),
 "approvedBy": ObjectId("64f91..."),
 "approvedAt": ISODate("2024-07-21T00:30:00Z")
}
```

certificates

```
{
 "_id": ObjectId("64f92..."),
 "applicationId": ObjectId("64f8f..."),
 "certificateUrl": "certificates/
  niramay_certificate.pdf",
 "issuedDate": ISODate("2024-07-15"),
 "issuedBy": ObjectId("64f91...")
}
```

Indexes

Collection	Index Fields	Type	Purpose
users	email	Unique	Ensures unique email for each user
applications	studentId, internshipId	Compound Unique	Prevent multiple applications for same internship
internships	companyId	Ascending	Faster search by company
applications	status	Ascending	Filter applications by status quickly
reports	applicationId	Ascending	Faster access to reports of an application

Relationships Summary

- User 1 — 1 Student
- User 1 — 1 Company
- Company 1 — N Internships (posts)
- Student 1 — N Applications (submits)
- Internship 1 — N Applications (receives)
- Application 1 — 1 Report (has)
- Report 1 — 1 Certificate (generates)

Design Notes

- All collections include _id as primary key (ObjectId).
- Timestamps (createdAt, updatedAt) are used for tracking.
- References are stored using ObjectId to maintain relationships.
- File uploads (resume, report, certificate) are stored as file paths or URLs.
- Role-based access control will be implemented in application layer.

mongoDB.

Why MongoDB?

- Flexible Schema
- High Performance
- Easy Scalability
- Cost Effective
- Developer Friendly

6. Frontend Architecture

InternManage
Internship Management Portal

- Project Overview
- Frontend Architecture
- Folder Structure
- Routing Structure
- Component Hierarchy
- State Management
- UI/UX Design System
- API Integration
- Authentication Flow



Purpose

This architecture ensures a scalable, maintainable and performant frontend for the Internship Management Portal.

Frontend Architecture

Internship Management Portal

A scalable and maintainable frontend architecture built with React.js, ensuring reusability of components, separation of concerns, and smooth user experience across all devices.

Component Based
UI is broken into small, reusable and independent components.

Route Based
Pages are rendered based on protected and public routes.

State Managed
Global state is managed using Context API & useReducer.

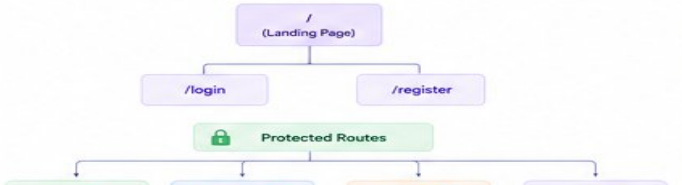
API Driven
All data is fetched from backend APIs using Axios.

Folder Structure

```
src/
  - assets/
      - images/
      - icons/
  - components/
      - common/
      - layout/
  - pages/
      - auth/
      - student/
      - company/
      - admin/
  - context/
  - hooks/
  - services/
  - utils/
  - App.jsx
  - main.jsx
```

assets/	- Images, icons, static files
components/	- Reusable UI components
pages/	- All page components
context/	- Global state and context
services/	- API calls and axios instance
utils/	- Helper functions and constants

2. Routing Structure (React Router DOM)



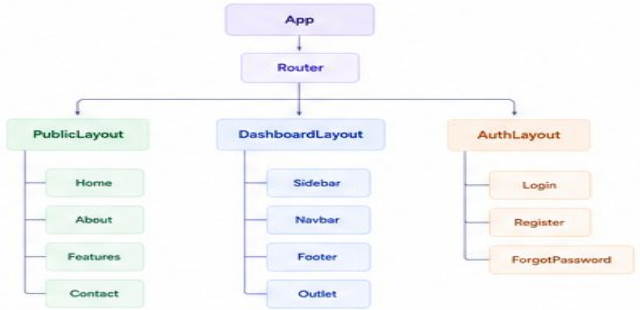
- /student/dashboard
- /student/internships
- /student/applications
- /student/profile
- /student/reports

- /company/dashboard
- /company/post
- /company/applications
- /company/interns
- /company/profile

- /admin/dashboard
- /admin/users
- /admin/companies
- /admin/internships
- /admin/reports

- /about
- /features
- /how-it-works
- /contact

3. Component Hierarchy (Example)



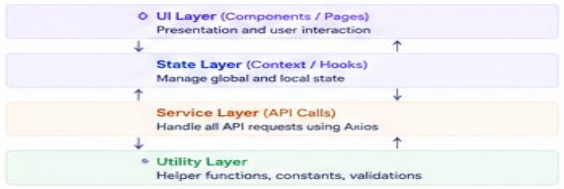
Outlet is used to render child routes inside DashboardLayout

4. State Management



useReducer is used for complex state logic.

5. Layered Architecture (High Level)



6. UI/UX Design System

Color Palette	
Typography	Inter (Primary Font)
Spacing	8px grid system
Border Radius	8px / 12px / 16px
Shadows	Soft and subtle shadows
Responsive Breakpoints	

7. API Integration



Features:
- Centralized Axios instance
- Interceptors for token
- Error handling
- Loading state management

8. Authentication Flow



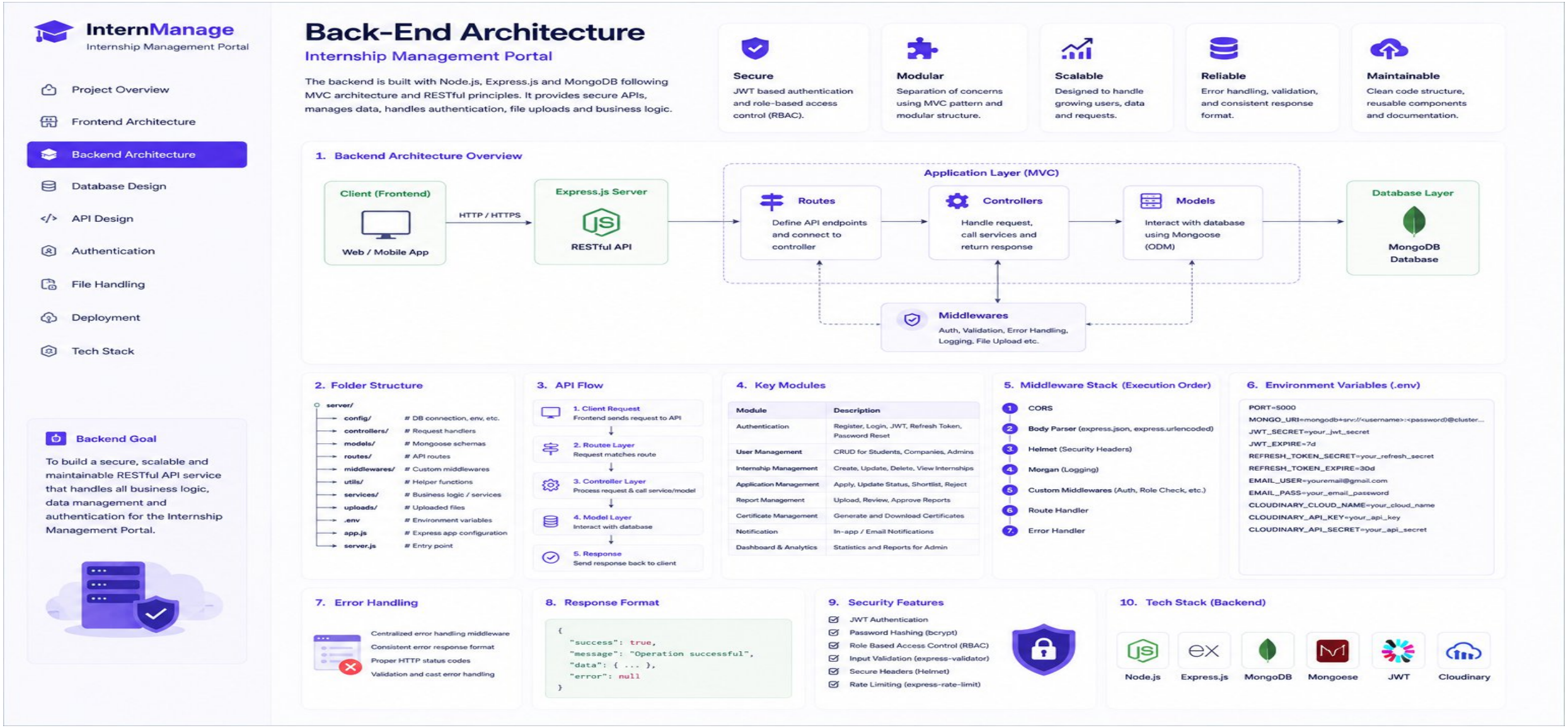
Features:
- JWT based authentication
- Protected routes
- Role based access (Student, Company, Admin)
- Auto logout on token expiry

9. Tech Stack (Frontend)

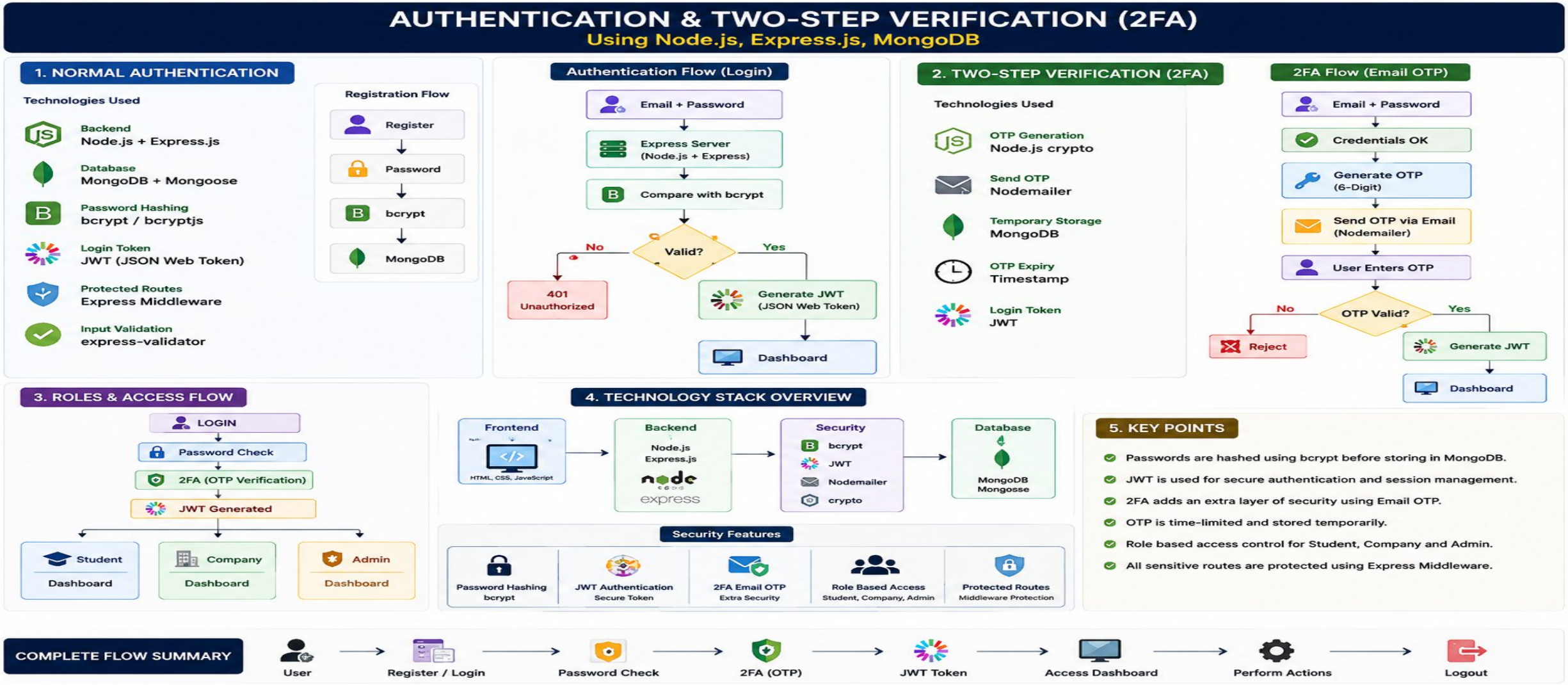


These technologies ensure performance, scalability and better developer experience.

7. Backend Architecture



8. Authentication & Two-Step Verification



9. CRUD Operations

CREATE

POST

Register user
Post internship
Submit application
Upload report

READ

GET

Browse internships
View applications
View reports
Admin analytics

UPDATE

PUT / PATCH

Edit profile
Edit internship
Change application status
Approve reports

DELETE

DELETE

Remove internship
Remove/disable records
Admin cleanup actions

10. Complete Internship Workflow

1. Student registers
2. Finds internship
3. Submits application
4. Company reviews
5. Accepted / rejected
6. Student completes internship
7. Report submitted
8. Admin verifies
9. Certificate issued

Core idea

The portal tracks the complete journey from internship discovery to verified completion.

11. Technology Stack

Frontend

HTML5
CSS3
JavaScript
Responsive UI
Fetch API

Backend

Node.js
Express.js
REST APIs
Routes
Controllers
Middleware

Database & Security

MongoDB
Mongoose
bcrypt
JWT
Role-based access
Validation

Tools

Git
GitHub
Postman
Multer
Nodemailer
Cloudinary (optional)

12. Current Status & Development Roadmap

Already Established

- ✓ Repository and project structure
- ✓ Express server setup
- ✓ MongoDB connection/configuration
- ✓ Routes / controllers / models organization
- ✓ Middleware structure
- ✓ Frontend project direction
- ✓ Database and architecture planning

Next Development Steps

1. Complete User model and registration
2. Implement bcrypt password hashing
3. Add login + JWT
4. Add role-based middleware
5. Implement internship CRUD
6. Build application workflow
7. Connect frontend to APIs
8. Add reports/certificates
9. Test and deploy

13. Conclusion

✓ The project provides a centralized solution for internship management.

✓ Its architecture separates frontend, API routing, business logic and database access.

✓ CRUD operations manage core internship data, while authentication protects access.

✓ MongoDB provides flexible storage for users, internships, applications, reports and certificates.

✓ The planned 2FA layer adds an additional security step.

✓ The project is intentionally being developed in stages so each part can be understood and tested.

Thank You ● Questions & Discussion